Hamming

Generated by Doxygen 1.9.8

1 Class Index 1
1.1 Class List . . . 1

2 File Index 3
2.1 File List . . . 3

3 Class Documentation 5
3.1 DataBuffer Class Reference . . . 5
3.1.1 Detailed Description . . . 6
3.1.2 Constructor & Destructor Documentation . . . 6
3.1.2.1 DataBuffer() [1/2] . . . 6
3.1.2.2 DataBuffer() [2/2] . . . 7
3.1.3 Member Function Documentation . . . 7
3.1.3.1 copy_from() . . . 7
3.1.3.2 get_bit() . . . 7
3.1.3.3 get_file_size() . . . 8
3.1.3.4 get_total_bits() . . . 8
3.1.3.5 print_bit_range() . . . 8
3.1.3.6 set_bit() . . . 9
3.1.4 Member Data Documentation . . . 9
3.1.4.1 __data . . . 9
3.1.4.2 __file_size . . . 9
3.1.4.3 __total_bits . . . 9
3.2 Hamming Class Reference . . . 10
3.2.1 Detailed Description . . . 10
3.2.2 Constructor & Destructor Documentation . . . 11
3.2.2.1 Hamming() . . . 11
3.2.3 Member Function Documentation . . . 11
3.2.3.1 decode() . . . 11
3.2.3.2 encode() . . . 11
3.2.3.3 show() . . . 12
3.2.4 Member Data Documentation . . . 12
3.2.4.1 __generator_matrix . . . 12
3.2.4.2 __parity_matrix . . . 12
3.3 Manager Class Reference . . . 13
3.3.1 Detailed Description . . . 13
3.3.2 Member Function Documentation . . . 13
3.3.2.1 calculate_ber() . . . 13
3.3.2.2 demo() . . . 14
3.3.2.3 extract_ber() . . . 14
3.4 NoiseChannel Class Reference . . . 14
3.4.1 Detailed Description . . . 15
3.4.2 Member Function Documentation . . . 15

3.4.2.1 transmit() 15

4 File Documentation **17**

4.1 src/DataBuffer/DataBuffer.hpp File Reference 17

4.2 DataBuffer.hpp 18

4.3 src/Hamming/Hamming.hpp File Reference 19

4.4 Hamming.hpp 20

4.5 src/main.cpp File Reference 22

4.5.1 Function Documentation 23

4.5.1.1 main() 23

4.6 main.cpp 23

4.7 src/Manager/Manager.hpp File Reference 24

4.8 Manager.hpp 25

4.9 src/NoiseChannel/NoiseChannel.hpp File Reference 27

4.10 NoiseChannel.hpp 27

Index **29**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

DataBuffer
Gerenciador de memória de alta performance para leitura de arquivos binários 5
Hamming
Responsável por implementar o Código de Hamming, seja na saída ou na recepção 10
Manager 13
NoiseChannel
Simula um canal de comunicação com ruído branco gaussiano aditivo (AWGN) 14

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

src/main.cpp 22
src/DataBuffer/DataBuffer.hpp 17
src/Hamming/Hamming.hpp 19
src/Manager/Manager.hpp 24
src/NoiseChannel/NoiseChannel.hpp 27

Chapter 3

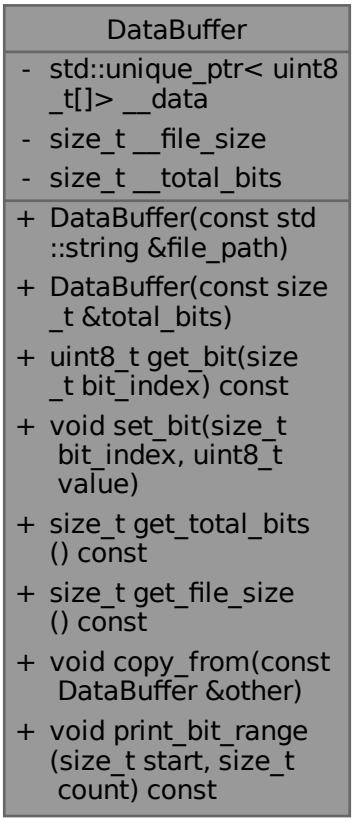
Class Documentation

3.1 DataBuffer Class Reference

Gerenciador de memória de alta performance para leitura de arquivos binários.

```
#include <DataBuffer.hpp>
```

Collaboration diagram for DataBuffer:



Public Member Functions

- DataBuffer (const std::string &file_path)

  Construtor que carrega um arquivo para a memória.
- DataBuffer (const size_t &total_bits)

  Construtor que aloca um espaço específico para um DataBuffer.
- uint8_t get_bit (size_t bit_index) const

  Recupera o valor de um bit específico na sequência global.
- void set_bit (size_t bit_index, uint8_t value)

  Define o valor de um bit específico.
- size_t get_total_bits () const

  Obtém o número total de bits carregados.
- size_t get_file_size () const

  Obtém o tamanho do arquivo original em bytes.
- void copy_from (const DataBuffer &other)

  Sobrescreve o conteúdo deste buffer com os dados de outro, sem realocar memória.
- void print_bit_range (size_t start, size_t count) const

  Imprime no console uma sequência de bits para fins de depuração.

Private Attributes

- std::unique_ptr< uint8_t[ ]> __data
- size_t __file_size
- size_t __total_bits

3.1.1 Detailed Description

Gerenciador de memória de alta performance para leitura de arquivos binários.

Esta classe carrega qualquer tipo de arquivo (.txt, .mp4, .png, etc.) para um buffer contíguo de bytes e permite o acesso individual a cada bit de forma otimizada.

Definition at line 16 of file DataBuffer.hpp.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 DataBuffer() [1/2]

```
DataBuffer::DataBuffer (
            const std::string & file_path )  [inline]
```

Construtor que carrega um arquivo para a memória.

Abre o arquivo em modo binário, calcula seu tamanho e aloca memória suficiente para armazenar o conteúdo completo.

Parameters

file_path	Caminho relativo ou absoluto do arquivo a ser lido.

Exceptions

std::runtime_error	Caso o arquivo não possa ser aberto ou lido.

Definition at line 31 of file DataBuffer.hpp.

3.1.2.2 DataBuffer() [2/2]

```
DataBuffer::DataBuffer (
            const size_t & total_bits )  [inline]
```

Construtor que aloca um espaço específico para um DataBuffer.

Parameters

total_bits	Total de Bits que será alocado.

Definition at line 59 of file DataBuffer.hpp.

3.1.3 Member Function Documentation

3.1.3.1 copy_from()

```
void DataBuffer::copy_from (
            const DataBuffer & other )  [inline]
```

Sobrescreve o conteúdo deste buffer com os dados de outro, sem realocar memória.

Parameters

other	O buffer de origem (deve ter o mesmo tamanho).

Definition at line 125 of file DataBuffer.hpp.

3.1.3.2 get_bit()

```
uint8_t DataBuffer::get_bit (
            size_t bit_index ) const  [inline]
```

Recupera o valor de um bit específico na sequência global.

Utiliza operações de bitwise (deslocamento e máscara) para extrair o bit sem a necessidade de converter o byte inteiro, garantindo performance O(1).

Parameters

bitIndex	O índice do bit desejado (0 até __total_bits - 1).

Returns

uint8_t Retorna 0 ou 1.

Exceptions

std::out_of_range	Se o índice solicitado for maior que o total de bits.

Definition at line 75 of file DataBuffer.hpp.

3.1.3.3 get_file_size()

```
size_t DataBuffer::get_file_size ( ) const  [inline]
```

Obtém o tamanho do arquivo original em bytes.

Returns

size_t Tamanho em bytes.

Definition at line 119 of file DataBuffer.hpp.

3.1.3.4 get_total_bits()

```
size_t DataBuffer::get_total_bits ( ) const  [inline]
```

Obtém o número total de bits carregados.

Returns

size_t Total de bits.

Definition at line 113 of file DataBuffer.hpp.

3.1.3.5 print_bit_range()

```
void DataBuffer::print_bit_range (
            size_t start,
            size_t count ) const  [inline]
```

Imprime no console uma sequência de bits para fins de depuração.

É garantido que não haverão erros no caso de count ser maior que o total de bits

Parameters

start	Índice inicial do bit.
count	Quantidade de bits a serem exibidos.

Definition at line 140 of file DataBuffer.hpp.

3.1.3.6 set_bit()

```
void DataBuffer::set_bit (
            size_t bit_index,
            uint8_t value ) [inline]
```

Define o valor de um bit específico.

Parameters

bit_index	Índice do bit.
value	Valor a ser definido (0 ou 1).

Definition at line 94 of file DataBuffer.hpp.

3.1.4 Member Data Documentation

3.1.4.1 __data

```
std::unique_ptr<uint8_t[]> DataBuffer::__data [private]
```

Smart Pointer para o buffer de bytes brutos.

Definition at line 18 of file DataBuffer.hpp.

3.1.4.2 __file_size

```
size_t DataBuffer::__file_size [private]
```

Tamanho total do arquivo em bytes.

Definition at line 19 of file DataBuffer.hpp.

3.1.4.3 __total_bits

```
size_t DataBuffer::__total_bits [private]
```

Quantidade total de bits disponíveis para processamento.

Definition at line 20 of file DataBuffer.hpp.

The documentation for this class was generated from the following file:

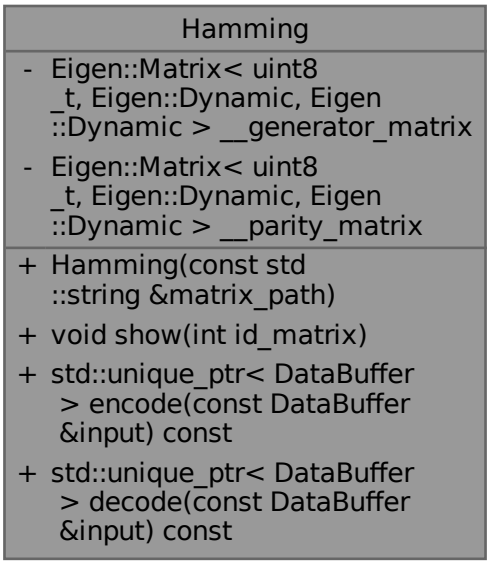
- src/DataBuffer/DataBuffer.hpp

3.2 Hamming Class Reference

Responsável por implementar o Código de Hamming, seja na saída ou na recepção.

```
#include <Hamming.hpp>
```

Collaboration diagram for Hamming:



Public Member Functions

- Hamming (const std::string &matrix_path)

  Construtor que carrega a matriz geradora a partir de um arquivo .txt.
- void show (int id_matrix)

  Apresentará as matrizes.
- std::unique_ptr< DataBuffer > encode (const DataBuffer &input) const

  Codifica um DataBuffer de entrada gerando um novo com palavras-código.
- std::unique_ptr< DataBuffer > decode (const DataBuffer &input) const

  Decodifica um buffer ruidoso, corrigindo erros de 1 bit por bloco.

Private Attributes

- Eigen::Matrix< uint8_t, Eigen::Dynamic, Eigen::Dynamic > __generator_matrix
- Eigen::Matrix< uint8_t, Eigen::Dynamic, Eigen::Dynamic > __parity_matrix

3.2.1 Detailed Description

Responsável por implementar o Código de Hamming, seja na saída ou na recepção.

Definition at line 16 of file Hamming.hpp.

3.2.2 Constructor & Destructor Documentation

3.2.2.1 Hamming()

```
Hamming::Hamming (
            const std::string & matrix_path )  [inline]
```

Construtor que carrega a matriz geradora a partir de um arquivo .txt.

O arquivo deve conter a matriz em formato de texto, ex: 1 0 0 0 1 1 0 0 1 0 0 1 0 1 ...

Parameters

matrix_path	Caminho para o arquivo .txt da matriz.

Definition at line 31 of file Hamming.hpp.

3.2.3 Member Function Documentation

3.2.3.1 decode()

```
std::unique_ptr< DataBuffer > Hamming::decode (
            const DataBuffer & input ) const  [inline]
```

Decodifica um buffer ruidoso, corrigindo erros de 1 bit por bloco.

Parameters

input	Buffer contendo as palavras-código (n bits cada).

Returns

std::unique_ptr<DataBuffer> Dados originais recuperados (k bits cada).

Se o erro estiver entre k e n, significa que o erro ocorreu em um bit de paridade. Como só nos interessamos pelos bits de dados (0 a k-1), esses erros são corrigidos "silenciosamente" apenas por não serem incluídos na extração.

Definition at line 142 of file Hamming.hpp.

3.2.3.2 encode()

```
std::unique_ptr< DataBuffer > Hamming::encode (
            const DataBuffer & input ) const  [inline]
```

Codifica um DataBuffer de entrada gerando um novo com palavras-código.

Parameters

input	Referência para o data_buffer contendo os dados originais.

Returns

std::unique_ptr<data_buffer> Ponteiro para o novo buffer codificado.

Definition at line 102 of file Hamming.hpp.

3.2.3.3 show()

```
void Hamming::show (
            int id_matrix )  [inline]
```

Apresentará as matrizes.

Parameters

id_matrix	Positivo mostra Matriz Geradora. Negativo mostra Matriz de Paridade.

Definition at line 91 of file Hamming.hpp.

3.2.4 Member Data Documentation

3.2.4.1 __generator_matrix

```
Eigen::Matrix<uint8_t, Eigen::Dynamic, Eigen::Dynamic> Hamming::__generator_matrix  [private]
```

Matriz G (k x n).

Definition at line 18 of file Hamming.hpp.

3.2.4.2 __parity_matrix

```
Eigen::Matrix<uint8_t, Eigen::Dynamic, Eigen::Dynamic> Hamming::__parity_matrix  [private]
```

Definition at line 19 of file Hamming.hpp.

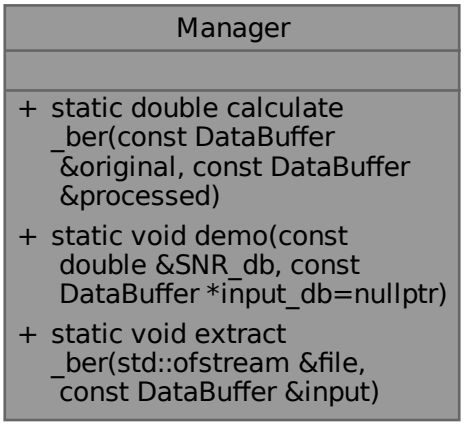
The documentation for this class was generated from the following file:

- src/Hamming/Hamming.hpp

3.3 Manager Class Reference

```
#include <Manager.hpp>
```

Collaboration diagram for Manager:



Static Public Member Functions

- static double calculate_ber (const DataBuffer &original, const DataBuffer &processed)

  Compara dois buffers bit a bit, conta o total de divergências e calcula a BER.
- static void demo (const double &SNR_db, const DataBuffer *input_db=nullptr)

  Executa uma simulação completa de transmissão de dados via código de Hamming.
- static void extract_ber (std::ofstream &file, const DataBuffer &input)

3.3.1 Detailed Description

Definition at line 11 of file Manager.hpp.

3.3.2 Member Function Documentation

3.3.2.1 calculate_ber()

```
static double Manager::calculate_ber (
            const DataBuffer & original,
            const DataBuffer & processed )  [inline], [static]
```

Compara dois buffers bit a bit, conta o total de divergências e calcula a BER.

Útil para calcular a taxa de erro de bit (BER) após a transmissão ou decodificação.

Parameters

original	Buffer com os dados de referência (originais).
processed	Buffer com os dados que passaram pelo canal e pelo decoder.

Returns

double BER

Definition at line 21 of file Manager.hpp.

3.3.2.2 demo()

```
static void Manager::demo (
            const double & SNR_db,
            const DataBuffer * input_db = nullptr )  [inline], [static]
```

Executa uma simulação completa de transmissão de dados via código de Hamming.

Esta função realiza o fluxo fim-a-fim: exibição dos dados originais, codificação Hamming, simulação de ruído branco (AWGN) no canal e decodificação para tentativa de recuperação dos bits. Se nenhum buffer for fornecido, a função executa um "caso padrão" de 4 bits (0010).

Parameters

SNR_db	Relação Sinal-Ruído (SNR) em decibéis para a simulação do canal.
custom_db	Ponteiro opcional para um DataBuffer. Se for nullptr, usa o buffer padrão.

Definition at line 49 of file Manager.hpp.

3.3.2.3 extract_ber()

```
static void Manager::extract_ber (
            std::ofstream & file,
            const DataBuffer & input )  [inline], [static]
```

Definition at line 85 of file Manager.hpp.

The documentation for this class was generated from the following file:

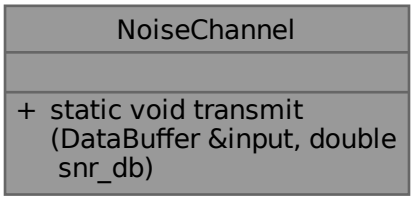
- src/Manager/Manager.hpp

3.4 NoiseChannel Class Reference

Simula um canal de comunicação com ruído branco gaussiano aditivo (AWGN).

```
#include <NoiseChannel.hpp>
```

Collaboration diagram for NoiseChannel:



Static Public Member Functions

- static void transmit (DataBuffer &input, double snr_db)

  Aplica ruído AWGN diretamente no buffer recebido.

3.4.1 Detailed Description

Simula um canal de comunicação com ruído branco gaussiano aditivo (AWGN).

Definition at line 12 of file NoiseChannel.hpp.

3.4.2 Member Function Documentation

3.4.2.1 transmit()

```
static void NoiseChannel::transmit (
            DataBuffer & input,
            double snr_db )  [inline], [static]
```

Aplica ruído AWGN diretamente no buffer recebido.

Parameters

out	*buffer*	Ponteiro único para o buffer de dados que será modificado.
	snr_db	Relação sinal-ruído em dB.

Definition at line 20 of file NoiseChannel.hpp.

The documentation for this class was generated from the following file:

- src/NoiseChannel/NoiseChannel.hpp

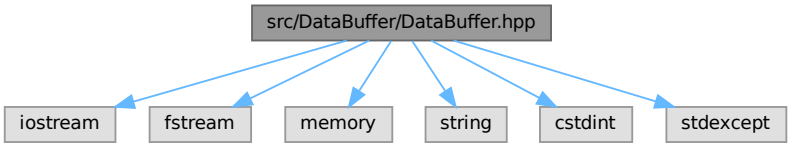
Chapter 4

File Documentation

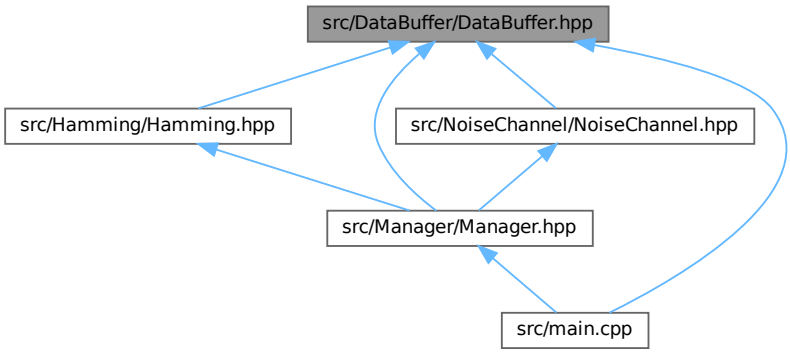
4.1 src/DataBuffer/DataBuffer.hpp File Reference

```
#include <iostream>
#include <fstream>
#include <memory>
#include <string>
#include <cstdint>
#include <stdexcept>
```

Include dependency graph for DataBuffer.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class DataBuffer

  Gerenciador de memória de alta performance para leitura de arquivos binários.

4.2 DataBuffer.hpp

Go to the documentation of this file.

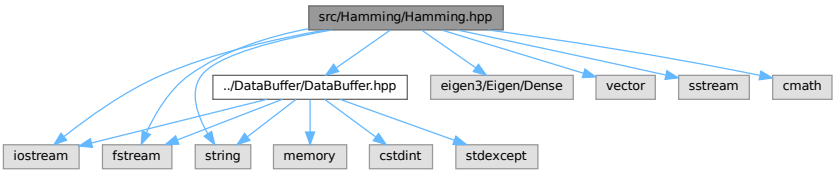
```
#pragma once

#include <iostream>
#include <fstream>
#include <memory>
#include <string>
#include <cstdint>
#include <stdexcept>

class DataBuffer {
private:
    std::unique_ptr<uint8_t[]> __data;
    size_t __file_size;
    size_t __total_bits;
public:
    DataBuffer(const std::string& file_path) {
        std::ifstream file(
            file_path,
            // Abrimos de forma binária e colocamos o ponteiro no fim do arquivo
            std::ios::binary | std::ios::ate
        );

        if (!file.is_open()) {
            throw std::runtime_error("Erro: Falha ao abrir o arquivo especificado.");
        }

        this->__file_size = file.tellg();
        this->__total_bits = this->__file_size * 8;

        // Alocação de memória bruta para evitar overhead de containers complexos
        this->__data = std::make_unique<uint8_t[]>(this->__file_size);

        // Movemos o ponteiro de volta ao início
        file.seekg(0, std::ios::beg);
        // Carregamos o arquivo inteiro na RAM <-- CUIDADO
        file.read(reinterpret_cast<char*>(this->__data.get()), this->__file_size);
        file.close();
    }

    DataBuffer(const size_t& total_bits) {
        this->__total_bits = total_bits;
        this->__file_size = (total_bits + 7) / 8;
        this->__data = std::make_unique<uint8_t[]>(this->__file_size);
        // Não preencheremos porque julgo não necessário
    }

    inline uint8_t get_bit(size_t bit_index) const {
        if (bit_index >= this->__total_bits) {
            throw std::out_of_range("Erro: Índice de bit fora dos limites do buffer.");
        }

        // byte_idx identifica em qual byte o bit está
        size_t byte_idx = bit_index / 8;

        // bit_pos identifica a posição dentro do byte (da esquerda para a direita / MSB)
        uint8_t bit_pos = 7 - (bit_index % 8);

        return (this->__data[byte_idx] » bit_pos) & 1;
    }

    inline void set_bit(size_t bit_index, uint8_t value) {
        if (bit_index >= this->__total_bits) {
            return;
        }

        size_t byte_idx = bit_index / 8;
        uint8_t bit_pos = 7 - (bit_index % 8);

        if (value) {
            this->__data[byte_idx] |= (1 « bit_pos);  // Define como 1
        } else {
```

```
            this->__data[byte_idx] &= ~(1 « bit_pos); // Define como 0
        }
    }

    size_t get_total_bits() const { return this->__total_bits; }

    size_t get_file_size() const { return this->__file_size; }

    void copy_from(const DataBuffer& other){
        std::copy(
            other.__data.get(),
            other.__data.get() + other.__file_size,
            this->__data.get()
        );
    }

    void print_bit_range(size_t start, size_t count) const {
        for (size_t i = start; i < start + count && i < this->__total_bits; ++i) {
            std::cout « static_cast<int>(this->get_bit(i));
        }
        std::cout « std::endl;
    }
};
```

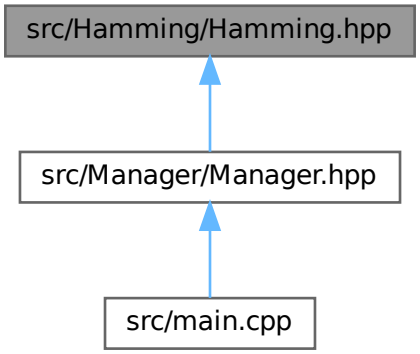
4.3 src/Hamming/Hamming.hpp File Reference

```
#include <iostream>
#include <eigen3/Eigen/Dense>
#include <fstream>
#include <string>
#include <vector>
#include <sstream>
#include <cmath>
#include "../DataBuffer/DataBuffer.hpp"
```

Include dependency graph for Hamming.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class Hamming

  Responsável por implementar o Código de Hamming, seja na saída ou na recepção.

4.4 Hamming.hpp

Go to the documentation of this file.

```
#pragma once

#include <iostream>
#include <eigen3/Eigen/Dense>
#include <fstream>
#include <string>
#include <vector>
#include <sstream>
#include <cmath>

#include "../DataBuffer/DataBuffer.hpp"

class Hamming {
private:
    Eigen::Matrix<uint8_t, Eigen::Dynamic, Eigen::Dynamic> __generator_matrix;
    Eigen::Matrix<uint8_t, Eigen::Dynamic, Eigen::Dynamic> __parity_matrix;

public:
    Hamming(const std::string& matrix_path) {
        std::ifstream file(matrix_path);
        if (!file.is_open()) {
            throw std::runtime_error("Erro ao abrir matriz geradora.");
        }

        /* Determinação de Dimensões */
        size_t rows = 0;
        size_t cols = 0;
        std::string line;

        if(std::getline(file, line)){
            rows++;
            std::stringstream ss(line);
            int temp_val;
            while(ss » temp_val){
                cols++;
            }
        }
```

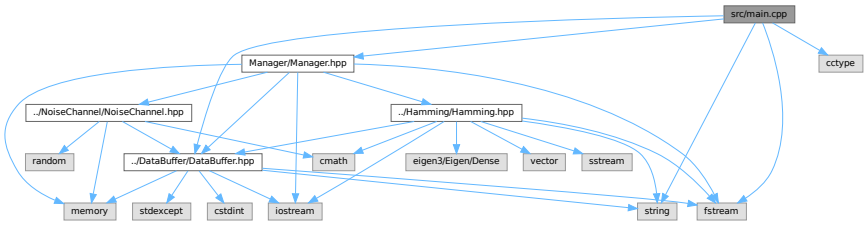
```
        while(std::getline(file, line)){
            if (!line.empty()) {
                rows++;
            }
        }

        /* Atribuição da Matriz Geradora */
        this->__generator_matrix.resize(rows, cols);
        file.clear();
        file.seekg(0, std::ios::beg);
        for(size_t i = 0; i < rows; ++i){
            for(size_t j = 0; j < cols; ++j){
                // Lemos como inteiro, mas salvaremos como uint8_t
                int val;
                if(!(file » val)){
                    throw std::runtime_error("Erro: Matriz com formato inconsistente.");
                }
                this->__generator_matrix(i, j) = static_cast<uint8_t>(val);
            }
        }

        /* Atribuição da Matriz de Paridade */
        size_t m = cols - rows;
        this->__parity_matrix.resize(m, cols);

        for(size_t i = 0; i < m; ++i){
            for(size_t j = 0; j < rows; ++j){
                this->__parity_matrix(i, j) = this->__generator_matrix(j, rows + i);
            }
            for(size_t j = rows; j < cols; j++){
                this->__parity_matrix(i, j) = ((j - rows) == i) ? 1 : 0;
            }
        }

        file.close();
    }

    void show(int id_matrix){
        std::cout «
                  ((id_matrix > 0) ? this->__generator_matrix : this->__parity_matrix)
                  « std::endl;
    }

     std::unique_ptr<DataBuffer> encode(const DataBuffer& input) const {
        const size_t k_bits = static_cast<size_t>(this->__generator_matrix.rows());
        const size_t n_bits = static_cast<size_t>(this->__generator_matrix.cols());

        size_t total_input_bits = input.get_total_bits();
        size_t numblocks = (total_input_bits + k_bits - 1) / k_bits;
        size_t total_output_bits = numblocks * n_bits;

        std::unique_ptr<DataBuffer> output = std::make_unique<DataBuffer>(total_output_bits);

        for(size_t b = 0; b < numblocks; ++b) {
            // Para cada coluna 'j' da matriz geradora (cada bit da palavra-código de saída)
            for(size_t j = 0; j < n_bits; ++j) {
                int accumulator = 0;

                // Multiplicação da linha de dados pela coluna 'j' da matriz G
                for(size_t i = 0; i < k_bits; ++i) {
                    size_t bit_idx = b * k_bits + i;

                    // Pegamos o bit "on-the-fly" do buffer original
                    uint8_t data_bit = (bit_idx < total_input_bits) ? input.get_bit(bit_idx) : 0;

                    // Operação de produto: data_bit * G(i, j)
                    // Como data_bit é 0 ou 1, isso equivale a um AND lógico ou uma soma condicional
                    accumulator += data_bit * this->__generator_matrix(i, j);
                }

                // Aplica o módulo 2 (GF2) e seta o bit diretamente no destino
                output->set_bit(b * n_bits + j, static_cast<uint8_t>(accumulator % 2));
            }
        }

        return output;
     }

     std::unique_ptr<DataBuffer> decode(const DataBuffer& input) const {
        const size_t n = static_cast<size_t>(this->__parity_matrix.cols());
        const size_t m = static_cast<size_t>(this->__parity_matrix.rows());
        const size_t k = static_cast<size_t>(this->__generator_matrix.rows());

        const size_t total_input_bits = input.get_total_bits();
        const size_t numblocks = total_input_bits / n;

        std::unique_ptr<DataBuffer> output = std::make_unique<DataBuffer>(numblocks * k);
```

```
        /* Para posterior uso, note como já fizemos a alocação */
        std::vector<uint8_t> syndrome(m, 0);
        for(size_t b = 0; b < numblocks; b++){

            /* Cálculo da Síndrome */
            for(size_t i = 0; i < m; ++i){
                int acc = 0;
                for(size_t j = 0; j < n; ++j){
                    // Multiplicação bit a bit
                    acc += input.get_bit(b * n + j) * this->__parity_matrix(i , j);
                }
                syndrome[i] = static_cast<uint8_t>(acc % 2);
            }

            /* Verificação se há erro */
            int error_bit_index = -1; // -1 Significará que não há erro
            for(const uint8_t& s_bit : syndrome){
                if(s_bit != 0){ error_bit_index = 0; break; } // Então levamos à 0 para identificar
que houve erro
            }

            if(!error_bit_index){
                // (Propriedade)
                /* Se a síndrome não é zero, ela deve ser idêntica a uma das colunas de H*/
                for(size_t col = 0; col < n; ++col){
                    bool match = true;
                    for(size_t row = 0; row < m; ++row){
                        if(this->__parity_matrix(row, col) != syndrome[row]){
                            match = false;
                            break;
                        }
                    }

                    if(match){
                        error_bit_index = static_cast<int>(col);
                        break;
                    }
                }
            }

            /* Correção e Extração de Dados */
            for(size_t i = 0; i < k; ++i){
                uint8_t bit = input.get_bit(b * n + i);

                if(error_bit_index == static_cast<int>(i)){
                    bit = (bit == 1) ? 0 : 1;
                }

                output->set_bit(b * k + i, bit);
            }

        }

        return output;
    }
};
```

4.5 src/main.cpp File Reference

```
#include "Manager/Manager.hpp"
#include "DataBuffer/DataBuffer.hpp"
#include <string>
#include <fstream>
#include <cctype>
```

Include dependency graph for main.cpp:



Functions

- int main (int argc, char *argv[])

4.5.1 Function Documentation

4.5.1.1 main()

```
int main (
            int argc,
            char * argv[] )
```

Definition at line 10 of file main.cpp.

4.6 main.cpp

Go to the documentation of this file.

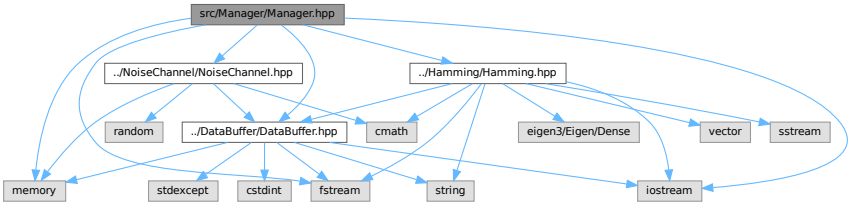
```
#include "Manager/Manager.hpp"
#include "DataBuffer/DataBuffer.hpp"

// Para tratarmos as entradas
#include <string>
#include <fstream>
#include <cctype>

// Alterar nome da variável para buffer_size
int main(int argc, char* argv[]){

    // Para verificarmos se há um arquivo com o nome dado
    auto is_file = [](const std::string& p) { return std::ifstream(p).good(); };
    auto is_number = [](const std::string& s) -> bool {
        try {
            size_t pos;
            std::stod(s, &pos);
            return pos == s.length();
        }  catch (...) {
            return false;
        }
    };
    switch(argc){
        case 1: {
            std::cout « "No parameters provided, assuming SNR_db = 1000.\n\n";
        }
        case 2: {
            Manager::demo(
                (argc == 2) ? std::stod(argv[1]) : 1000
            );
            return 0;
        }

```

```
        case 3: {
            /*
            Haverá Mais de Uma Possibilidade de Entrada Aqui
            $0 ./main

            Caso 1:
                $1 SNR_db
                $2 Nome do arquivo de entrada ou Sequência de Bits

            Caso 2:
                $1 Nome de um arquivo para expelirmos os resultados
                $2 Nome do arquivo de entrada
            */

            /* Verificação do Primeiro Argumento */
            if(!is_number(argv[1])){
                // Caso 2

                std::ofstream file(argv[1]);
                if(!file.is_open()){ std::cout « "Error in the exit file" « std::endl; return -1;}
                DataBuffer db(argv[2]);
                Manager::extract_ber(file, db);
                file.close();
                return 0;
            }

            // Caso 1

            // Entraremos com SNR_db e ou sequência de bits ou nome do arquivo a ser testado
            if(is_file(argv[2])){
                // Caso seja um arquivo válido
                DataBuffer db(argv[2]);
                Manager::demo(
                   std::stod(argv[1]),
                   &db
                );
                return 0;
            }

            // Caso não seja um arquivo válido, deve ser uma sequência de bits
            std::string bit_sequence = argv[2];
            size_t i = 0;
            size_t total_bits = bit_sequence.size();
            DataBuffer db(total_bits);
            while(i < total_bits){
                char bit_to_be_inserted = bit_sequence[i];

                if(
                    !std::isdigit(bit_to_be_inserted) || (bit_to_be_inserted != '0' &&
bit_to_be_inserted != '1')
                ){
                    std::cout « "Error in the bit sequence." « std::endl;
                    return -1;
                }
                db.set_bit(i, static_cast<uint8_t>(bit_to_be_inserted - 48));
                i++;
            }

            Manager::demo(
                std::stod(argv[1]),
                &db
            );
            return 0;
        }

    }

    return 0;
}
```

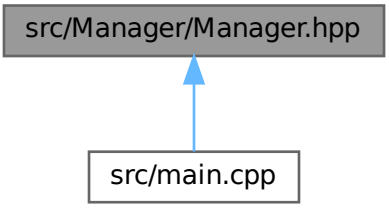
4.7 src/Manager/Manager.hpp File Reference

```
#include <iostream>
#include <fstream>
#include <memory>
#include "../DataBuffer/DataBuffer.hpp"
#include "../NoiseChannel/NoiseChannel.hpp"
```

`#include "../Hamming/Hamming.hpp"`
Include dependency graph for Manager.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class Manager

4.8 Manager.hpp

Go to the documentation of this file.

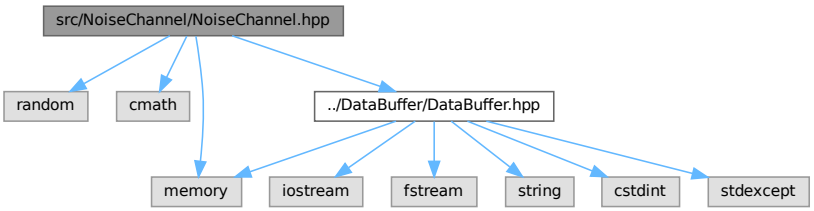
```
#pragma once

#include <iostream>
#include <fstream>
#include <memory>

#include "../DataBuffer/DataBuffer.hpp"
#include "../NoiseChannel/NoiseChannel.hpp"
#include "../Hamming/Hamming.hpp"

class Manager {
public:
    static double calculate_ber(const DataBuffer& original, const DataBuffer& processed) {
//        Acredito que não será necessário, dado nosso contexto
//        if (original.get_total_bits() != processed.get_total_bits()) {
//            throw std::invalid_argument("Erro: Buffers de tamanhos diferentes não podem ser comparados.");
//        }

        double error_count = 0;
        double total_bits = original.get_total_bits();

        for (size_t i = 0; i < total_bits; ++i) {
            if (original.get_bit(i) != processed.get_bit(i)) {
```

```
                error_count++;
            }
        }

        return error_count / total_bits;
    }

    static void demo(const double& SNR_db, const DataBuffer* input_db = nullptr){

        DataBuffer default_db(4);
        if(input_db == nullptr){
            default_db.set_bit(0, 0);
            default_db.set_bit(1, 0);
            default_db.set_bit(2, 1);
            default_db.set_bit(3, 0);
        }

        const DataBuffer& db = (input_db == nullptr) ? default_db : *input_db;

        std::cout « "Enviei: ";
                db.print_bit_range(0, db.get_total_bits()); // Apenas para garantirmos que todos os bits serão exibidos
        std::cout « std::endl;

        Hamming h("key.txt");

        auto sended_bits = h.encode(db);
        std::cout « "Codifiquei: ";
                sended_bits->print_bit_range(0, sended_bits->get_total_bits()); // Apenas para garantirmos que todos os bits serão exibidos
        std::cout « std::endl;

        NoiseChannel::transmit(*sended_bits, SNR_db);
        std::cout « "Após Efeito de Canal: ";
                sended_bits->print_bit_range(0, sended_bits->get_total_bits()); // Apenas para garantirmos que todos os bits serão exibidos
        std::cout « std::endl;

        auto received_bits = h.decode(*sended_bits);
        std::cout « "Recebi: ";
                received_bits->print_bit_range(0, db.get_total_bits()); // Apenas para garantirmos que todos os bits serão exibidos
        std::cout « std::endl;

        std::cout « "BER -> " « Manager::calculate_ber(db, *received_bits) « std::endl;
    }

    static void extract_ber(std::ofstream& file, const DataBuffer& input){

        const int SNR_DB_MAX = 10;
        const int ITERATIONS = 100;


        // Independe dos processos sucessivos
        Hamming h("key.txt");
        std::unique_ptr<DataBuffer> sended_bits = h.encode(input);

        DataBuffer noisy_work_buffer(sended_bits->get_total_bits());

        file « "SNR_DB,BER\n";
        for(double SNR_DB = 0; SNR_DB < SNR_DB_MAX; SNR_DB += 0.25){
            double mean_ber = 0;

            for(int i = 0; i < ITERATIONS; ++i){

                noisy_work_buffer.copy_from(*sended_bits);

                // Aplicamos o ruído sempre à sequência original
                NoiseChannel::transmit(noisy_work_buffer, SNR_DB);

                std::unique_ptr<DataBuffer> received_bits = h.decode(noisy_work_buffer);

                mean_ber += Manager::calculate_ber(input, *received_bits);
            }

            mean_ber /= ITERATIONS;

            file « SNR_DB « "," « mean_ber « '\n';
        }
    }
};
```

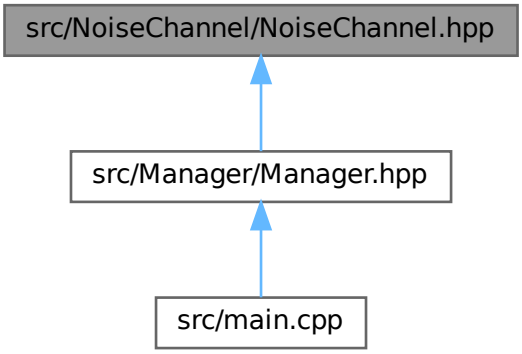
4.9 src/NoiseChannel/NoiseChannel.hpp File Reference

```
#include <random>
#include <cmath>
#include <memory>
#include "../DataBuffer/DataBuffer.hpp"
```

Include dependency graph for NoiseChannel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class NoiseChannel

  Simula um canal de comunicação com ruído branco gaussiano aditivo (AWGN).

4.10 NoiseChannel.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <random>
00004 #include <cmath>
```

```
#include <memory>

#include "../DataBuffer/DataBuffer.hpp"

class NoiseChannel {
public:

    static void transmit(DataBuffer& input, double snr_db) {
        static thread_local std::mt19937 __generator(std::random_device{}());

        size_t total_bits = input.get_total_bits();

        double snr_linear = std::pow(10, snr_db / 10.0);
        double sigma = std::sqrt(1.0 / snr_linear);

        std::normal_distribution<double> noise_dist(0.0, sigma);

        for(size_t i = 0; i < total_bits; ++i) {
            double signal = (input.get_bit(i) == 1) ? 1.0 : -1.0;
            double noisy_signal = signal + noise_dist(__generator);
            uint8_t received_bit = (noisy_signal > 0.0) ? 1 : 0;

            input.set_bit(i, received_bit);
        }
    }
};
```

Index

__data
    DataBuffer, 9
__file_size
    DataBuffer, 9
__generator_matrix
    Hamming, 12
__parity_matrix
    Hamming, 12
__total_bits
    DataBuffer, 9

calculate_ber
    Manager, 13
copy_from
    DataBuffer, 7

DataBuffer, 5
    __data, 9
    __file_size, 9
    __total_bits, 9
    copy_from, 7
    DataBuffer, 6, 7
    get_bit, 7
    get_file_size, 8
    get_total_bits, 8
    print_bit_range, 8
    set_bit, 9
decode
    Hamming, 11
demo
    Manager, 14

encode
    Hamming, 11
extract_ber
    Manager, 14

get_bit
    DataBuffer, 7
get_file_size
    DataBuffer, 8
get_total_bits
    DataBuffer, 8

Hamming, 10
    __generator_matrix, 12
    __parity_matrix, 12
    decode, 11
    encode, 11
    Hamming, 11
    show, 12

main
    main.cpp, 23
main.cpp
    main, 23
Manager, 13
    calculate_ber, 13
    demo, 14
    extract_ber, 14

NoiseChannel, 14
    transmit, 15

print_bit_range
    DataBuffer, 8

set_bit
    DataBuffer, 9
show
    Hamming, 12
src/DataBuffer/DataBuffer.hpp, 17, 18
src/Hamming/Hamming.hpp, 19, 20
src/main.cpp, 22, 23
src/Manager/Manager.hpp, 24, 25
src/NoiseChannel/NoiseChannel.hpp, 27

transmit
    NoiseChannel, 15